

1)What Is Application

An application is any program that helps a user complete a task, whether it's work, communication, learning, or entertainment.

We can build Application Using

- 1)Java Edition
- 2)Java Enterprise edition
- 3)Mobile/Micro Edition

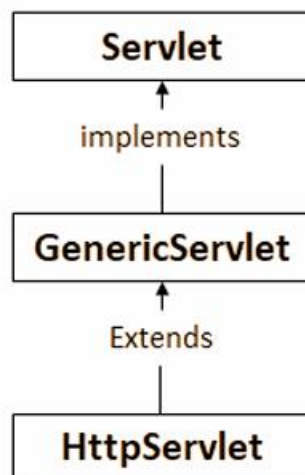
2)What is servlet

Servlet is an API which is used to develop web Applications

A Servlet is a Java class used to handle client requests and generate dynamic responses in web applications.

3)Ways to create Servlet:

- By implementing Servlet interface
- Extending GenericServlet
- Extending HttpServlet (most commonly used)



4)What Is Generic Servlet

GenericServlet is an **abstract class** that implements the Servlet interface and provides default implementations for most servlet methods.

5)What is httpServlet

HttpServlet is a class that extends GenericServlet and provides HTTP-specific methods such as:

- doGet()
- doPost()
- doPut()
- doDelete()

These methods handle different types of HTTP requests sent by the browser.

6)What is PrintWriter

- PrintWriter is a Java class used to **display or write data**.
- In Servlets, it is mainly used to send data from the server to the browser.

7)What is getWriter()?

- getWriter() is a method of HttpServletResponse.
- It returns a PrintWriter object used to send output to the browser.

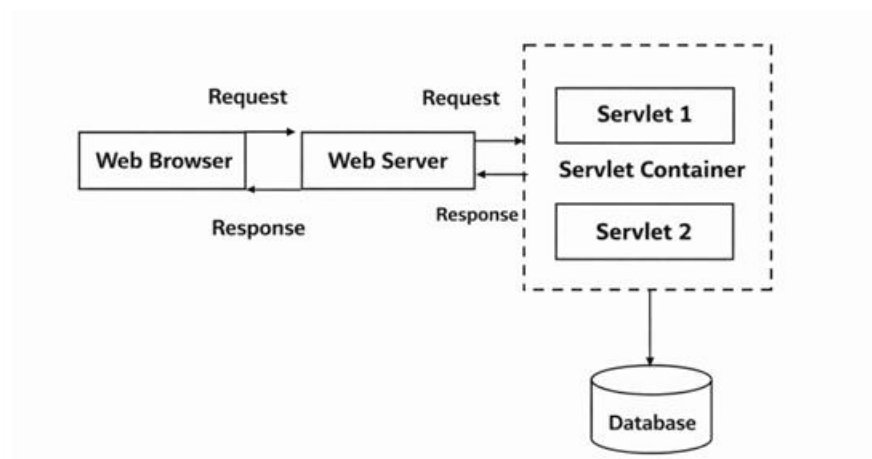
8)What is getParameter()?

- getParameter() is a method of HttpServletRequest.
- It is used to fetch data entered by the user in HTML forms.

9)Explain Servlet Life Cycle and its methods ?

- Loading → As per the request particular servlet class will be loaded to container
- Instantiation → The web container will create an instance for that loaded class
- Initialization → Called once when servlet is created. init()
- Request handling → Handles every request. service()
- Destruction → Called once before servlet is destroyed. destroy()

10)Servlet Architecture



11)Difference Between doGet() and doPost()

doGet()	doPost()
Used to get data from server	Used to send data to server
Data will be display in url	Data will not be display in url
Less secure	More secure
Faster	Bit slower

12)What is Request Dispatcher

Request Dispatcher is a interface it helps to dispatching the request from one servlet to another servlet/jsp/html or from one jsp to another jsp/servlet.

There are two methods are present in Request Dispatcher

1)include ()

2)forward()

1)forward():- when we want to transfer complete control to another resource then we make use of forward()

2)include():- when we want to add output of another resource into the current response and continue the execution then we will make use of include()

13) What is JSP and Life Cycle of JSP?

JSP (Java Server Pages) is used to create dynamic web pages using HTML and Java.

Life Cycle:

1. Translation (JSP → Servlet)
2. Compilation
3. Loading
4. Instantiation
5. Initialization → `jspInit()`
6. Request handling → `jspService()`
7. Destruction → `jspDestroy()`

14 Explain JSP Tags ?

1. Directive Tags () → Used to import the packages and to handling the exceptions.
2. Scriptlet Tags () → Used to write java code in jsp
3. Expression Tags () → Used to print the values or output
4. Declaration Tags () → Used to declare the class members
5. Action Tags () → Used to control the flow between the pages
6. Comment Tags () → Used to comment the code

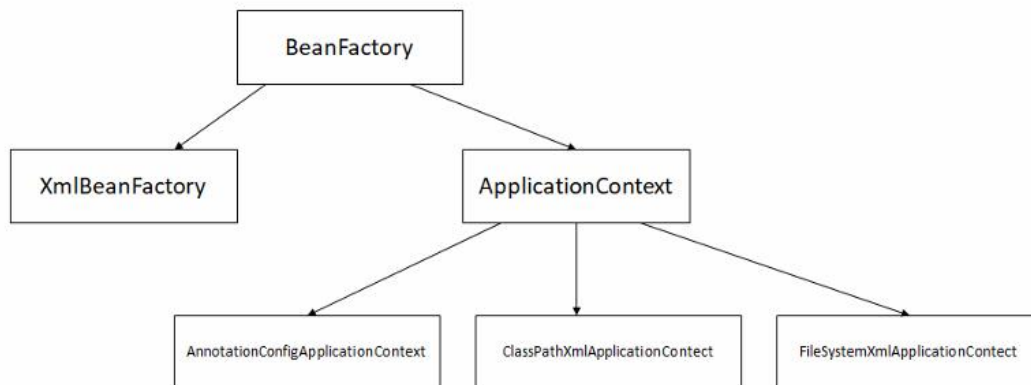
15)What Is Spring

Spring is a lightweight Java framework used for building enterprise applications with features like DI, AOP, MVC, etc. Spring internally works with hibernate It allows loose coupling Spring default follows Single-ton design pattern, which ensures only one bean is created. Spring follows Aspect oriented programming.

16)Names Of Spring Containers

- BeanFactory → Basic container
- ApplicationContext → Advanced container (most used)
- WebApplicationContext → In Spring MVC

Implementation classes for BeanFactory and ApplicationContext



17. What is Spring Container?

Spring container is the core part of Spring that creates, manages, and connects objects (beans) in a Spring application. It is responsible for: Creating objects (beans) Managing lifecycle Injecting dependencies

18. What is BeanFactory?

BeanFactory is the **basic container of Spring IoC** that:

- Creates objects
- Manages dependencies
- Provides lazy loading

19) What is XmlBeanFactory?

XmlBeanFactory is an old implementation of BeanFactory that reads XML configuration file.

20 Why is XmlBeanFactory deprecated?

Because:

- It is outdated
- Lacks modern features
- Replaced by ApplicationContext
- Spring encourages annotation-based configuration

21. What is ClassPathXmlApplicationContext?

It is an implementation of ApplicationContext that:

- Loads XML configuration from classpath
- Creates beans automatically

22 What is ApplicationContext?

ApplicationContext is an **advanced version of BeanFactory**.

It provides:

- Eager loading of beans
- Event handling
- AOP support
- Internationalization (i18n)
- Better enterprise features

23. What are Enterprise Applications?

Enterprise applications are large-scale software systems designed to support an organization's business processes and operations. They are scalable, reliable, and used by multiple users across departments. Example: Banking systems, e-commerce platforms

24. What is Spring Bean?

A Spring Bean is an **object that is created, configured, and managed by Spring IoC container**.

25. What is Inversion of Control (IOC)?

Spring IOC stands for Inversion of control, it is a design principle where the control of creating and managing the beans will be managed by spring container Object creation is handled by the container instead of the programmer

26. What is Dependency Injection (DI) & Types?

DI stands for Dependency Injection Dependency injection is a design pattern where injecting the objects of one class into another class externally is called as dependency injection We can't achieve DI without IOC

Types of dependency injection are:

1. Constructor Injection
2. Setter Injection
3. Variable or Field Injection

27. What happens when we call getBean()?

- Spring checks bean definition
- Creates object (if not already created)
- Injects dependencies
- Returns object

28 Bean Scope in Spring

- Only one object per Spring container by default singleton

29. Name some Spring Core Annotations?

- @Component -Used to create a Spring-managed bean automatically
- @Autowired -Used for Dependency Injection
- @Bean -Used to manually define a bean

Used inside @Configuration class

Useful when you don't have source code (external classes)

- @Configuration -Marks a class as configuration class , Replaces XML configuration
- @Scope -Defines bean scope (object creation behavior)
- @Primary-Gives highest priority when multiple beans exist
- @Qualifier-Used to select specific bean when multiple beans exist

Works with @Autowired

Solves ambiguity problem

31). What are Configuration Files?

Configuration Files used to define how objects (beans) are created, configured, and connected in a Spring application.

30)What is Spring MVC?

Spring MVC is a module of the Spring Framework that follows the Model-View-Controller architecture to develop web applications by separating data, business logic, and presentation.

Components: Model → Holds data (business logic, database interaction)

View → Displays data (JSP, HTML pages)

Controller → Handles user requests and controls flow

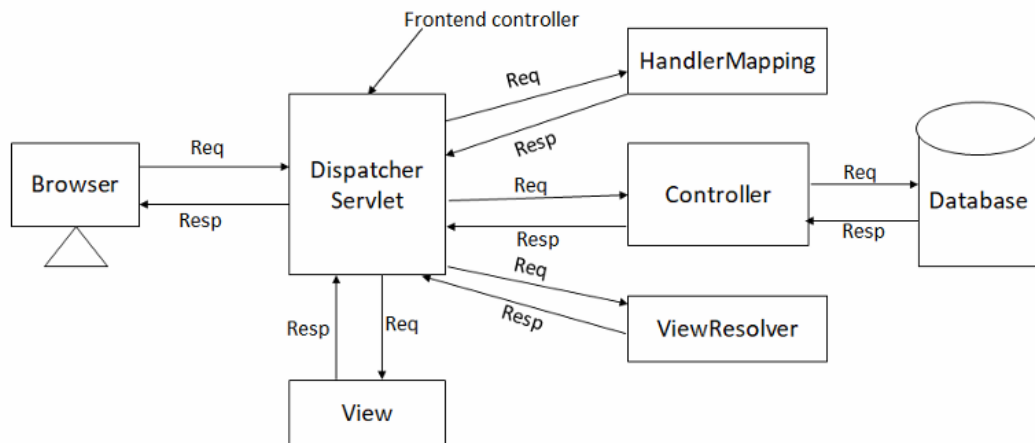
31)Manual Configuration in Spring MVC

Manual configuration means setting up Spring MVC without using auto-configuration. It involves:

- Configuring DispatcherServlet in web.xml to handle all incoming requests
- Creating a Spring configuration file (XML or Java-based) to define beans and enable MVC features
- Enabling component scanning to detect controllers
- Configuring a View Resolver to map logical view names to actual JSP pages
- Creating controller classes to handle requests and return views

32). Explain Spring MVC architecture?

- Spring MVC, part of the Spring Framework, follows a front controller architecture. When a request is sent from the frontend, it is first received by the DispatcherServlet, which acts as the central controller. The DispatcherServlet then consults the Handler Mapping to identify the appropriate Controller for the request.
- The request is forwarded to the Controller, which processes it by interacting with the Model (business logic and data). After processing, the Controller returns a ModelAndView object to the DispatcherServlet.
- The DispatcherServlet then passes the view name to the View Resolver, which resolves it into the actual view. Finally, the View is rendered and the response is sent back to the client



33). Explain Handler Mapping & View Resolver?

Handler Mapping is responsible for deciding which Controller should handle an incoming request. When a request reaches the Dispatcher Servlet, it asks the Handler Mapping to find the correct handler based on the URL or request details. The Handler Mapping then returns the appropriate Controller method that should process that request.

View Resolver, on the other hand, is responsible for deciding which view should be displayed to the user. After the Controller processes the request, it returns a logical view name (like “home” or “login”). The View Resolver converts this logical name into an actual view file, such as a JSP or HTML page.

34)What is Model,ModelMap,ModelAndView?

In Spring MVC, Model, ModelMap, and ModelAndView are used to send data from the controller to the view (JSP, Thymeleaf, etc.).

Model

- Model is an **interface** used to store data and send it to the view.
- Simple and commonly used
- Stores data using `addAttribute()`
- Returns only view name as String

ModelMap

- ModelMap is a **class** that works like a Map.
- Child class of LinkedHashMap
- Can use map operations
- Similar to Model
- Mostly used in older Spring MVC projects

ModelAndView

- ModelAndView is a **class** that contains:
 1. Model data
 2. View name
- Both are handled inside one object.
- Stores data and view together
- Useful when dynamically changing views
- More explicit approach

Important Spring MVC Annotations

1. @Controller

Used to make a class a Spring MVC Controller.

It handles client requests.

2. @RequestMapping

Used to map URL requests.

Can be used at:

- Class level
- Method level

3. @ModelAttribute

Used to bind form data with object.

35)Difference Between Spring Framework and Spring Boot

Spring	Spring Boot
Manual Configuration	Auto Configuration
Requires setup	Ready-to-run application
Needs External Server	Embedded Server
Add Dependencies Manually	Consists of starter files
We can develop web application	Develop web as well as Rest APIs

36). What is Spring Boot?

Spring Boot is a framework which has developed over Spring Framework that is used to develop standalone, production-ready Spring applications with minimal configuration. It simplifies the development process by providing auto-configuration, embedded servers, and default setups, so developers don't need to write extensive configuration code.

37). What are starter files? Name some starter files?

In Spring Boot starter files are predefined dependency packages that simplify project setup. A Spring Boot Starter is a collection of related dependencies that you can include in your project to enable a specific functionality without manually adding each library

Starter Files	Description
spring-boot-starter-web	Used to build web applications and REST APIs. Includes Spring MVC, embedded Tomcat, and JSON support.
spring-boot-starter-data-jpa	Used for database access using JPA and Hibernate. Provides repository support and simplifies CRUD operations.
spring-boot-devtools	Provides development-time features like auto restart. Helps in faster development with live reload support.
spring-boot-starter-security	Used to secure applications with authentication and authorization. Provides default login setup and security filters.
spring-boot-starter-thymeleaf	Used for server-side rendering of web pages using Thymeleaf templates. Integrates with Spring MVC for dynamic HTML
spring-boot-starter-validation	Used for validating user input using annotations like @NotNull, @Size. Ensures data correctness before processing

38) What is web services

Web services are a way for different software applications to communicate with each other over the internet.

A web service acts like a bridge between applications, allowing them to share data and functionality, even if they are built using different languages or run on different systems.

39) Types Of Web Services

SOAP (Simple Object Access Protocol)

- Uses XML
- More strict and secure
- Common in enterprise systems

REST (Representational State Transfer)

- Uses HTTP methods (GET, POST, PUT, DELETE)
- Lightweight and widely used
- Often returns JSON

40) What are REST APIs & Name some Rules of REST?

REST APIs (Representational State Transfer APIs) are web services that allow communication between client and server using standard HTTP methods. They follow a stateless architecture where data is exchanged in formats like JSON or XML, and each resource is identified by a unique URL.

- Stateless - Each request from the client must contain all the information needed. The server does not store client state between requests.
- Client–Server Architecture - The client and server are separate, allowing independent development and scalability. We should send the data in JSON format, because it is lightweight, simple, and JSON data can be directly converted into objects without complex parsing.
- HTTP Methods - For Proper HTTP request We should use proper HTTP methods
- The URL path should be independent We should follow proper HTTP protocol's The URL path shouldn't be verb and singular, it should be plural and noun.

41) Explain is spring boot thymleaf?

Spring Boot Thymeleaf is the combination of Spring Boot and Thymeleaf, where Thymeleaf acts as a server-side template engine to create dynamic HTML pages. It allows Spring Boot applications to render views by binding model data to HTML templates seamlessly.

42). What is Exception Handling and its types in spring boot?

Exception Handling in Spring Boot is the process of managing and responding to errors that occur during the execution of a Spring Boot application. It allows the application to handle runtime exceptions gracefully, provide meaningful error messages, and prevent the application from crashing.

Types:

1. Global Exception Handling

- @ControllerAdvice
- @ExceptionHandler
- @RestControllerAdvice

2. Controller Exception Handling Inside controller

3. Default error handling

43). Explain spring boot data validation ?

Spring Boot Data Validation is the process of ensuring that the data received from users or clients meets certain rules before being processed or stored. It helps prevent invalid or malicious data from entering the system, improving reliability and security

44)What is Spring Data Jpa

Spring Data JPA provides an easier way to implement:

- CRUD operations
- Database queries
- Pagination
- Sorting

using **JPA (Java Persistence API)** and repositories.

45)What is JpaRepository

JpaRepository is an interface provided by **Spring Data JPA** that contains ready-made methods for performing database operations.

It helps developers perform CRUD operations without writing SQL queries manually.

JpaRepository is an interface that:

- extends PagingAndSortingRepository
- which further extends CrudRepository

So it provides:

- CRUD operations
- Pagination
- Sorting
- JPA-specific features

46) Common Methods in JpaRepository

Method	Purpose	
save()	Insert/Update record	
findById()	Fetch record by ID	
findAll()	Fetch all records	
deleteById()	Delete by ID	
existsById()	Check record exists	
count()	Count records	

47)What is ResponseStructure

ResponseStructure is a custom wrapper class commonly used in Spring Boot applications to send responses in a proper structured format.

It is not a built-in class in Spring Boot.

Developers create it manually to standardize API responses.

Instead of returning raw data directly, we return:

- status code
- message
- actual data

48)What Is Optional

Optional is a class introduced in Java 8 to handle null values safely and avoid NullPointerException.

It is present in the java.util package.

49)Important Status Codes

- 1)200-Ok→Request Was Successful
- 2)201-Created→Resource created successfully. Used mainly with POST requests.
- 3) 202 -ACCEPTED→ Request accepted for processing.
- 4) 204 — >NO CONTENT
- 5) 301 — >MOVED PERMANENTLY ,URL permanently changed.
- 6) 400 — >BAD REQUEST Invalid client request.
- 7) 401 — >UNAUTHORIZED Authentication required.
- 8) 404 — >NOT FOUND Requested resource not found.
- 9) 405 — >METHOD NOT ALLOWED Wrong HTTP method used.
- 10) 500 — >INTERNAL SERVER ERROR Something went wrong on server side.

50)Layered Architecture Of Spring Boot

Layered Architecture in Spring Boot:

- divides application into multiple layers,
- improves code organization,
- separates business logic from database logic,
- makes enterprise applications easier to manage.

Main Layers in Spring Boot

Controller Layer

↓

Service Layer

↓

Repository (DAO) Layer

↓

Database

1. Controller Layer

Responsibility

Handles:

- client requests,
- HTTP methods,
- sending responses.

It acts as an entry point of the application.

Annotation Used

@RestController or @Controller

2. Service Layer

Responsibility

Contains:

- business logic,
- validations,
- processing logic.

Controller communicates with Service layer.

Annotation Used

@Service

3. Repository Layer (DAO Layer)

Responsibility

Handles database operations.

Communicates with database using:

- JPA,
- Hibernate,
- SQL.

Annotation Used

@Repository

Usually extends JpaRepository.

4. Database Layer

Stores application data.

Examples:

- MySQL
- Oracle
- PostgreSQL

Flow of Request

```
Client
↓
Controller
↓
Service
↓
Repository
↓
Database
```